DATA 412/612 · Statistical Programming in R · Week 8

8.3 dbplyr: Databases

A database keeps the tables on disk. dplyr states the request, dbplyr turns it into SQL, and collect() brings back only the rows you need.

1. What is a database?

A **database** is an organized collection of data, built for storage, retrieval, and management.

In R, a data frame lives in RAM. That is fine for `flights`. It is a poor plan when the table is larger than memory, when several people query it at once, or when the data already lives in a database. Loading every row into R can be slow or impossible.

A **relational database** splits the data into tables and connects them with keys. One soccer database might hold:

Table	What it stores
Country	Country names and ids
League	Soccer leagues
Match	Dates, scores, and the teams that played
Team	Team names and ids
Player	Player information

These tables stay together in one database. You ask for the rows you need. You do not load the whole database into R first.

---

2. What is SQLite?

SQLite is a lightweight relational database management system, a DBMS. It is the program that stores and queries the data.

`soccer.sqlite` is not the program. It is the **file** that holds the soccer tables. SQLite reads that file, runs queries, and returns rows.

One `.sqlite` file can hold many tables, plus indexes and other database information.

Other database systems include MySQL, MariaDB, PostgreSQL, Microsoft SQL Server, and Google BigQuery. The idea is the same. The program that talks to each one from R is different.

---

3. The packages, and what each one does

These pieces are easy to mix up. Each one has one job.

Piece	What it is
R / RStudio	Where you write and run R code
dplyr	The verbs: `filter()`, `select()`, `mutate()`, and the rest
dbplyr	Translates those verbs into SQL
SQL	The language a relational database understands
DBI	A standard interface between R and database systems
RSQLite	The DBI backend for SQLite
SQLite	The program that runs the SQL and manages the file
soccer.sqlite	The file that holds the tables

`dbplyr`, `DBI`, and `RSQLite` do **not** load with `library(tidyverse)`. Install them, then load them.

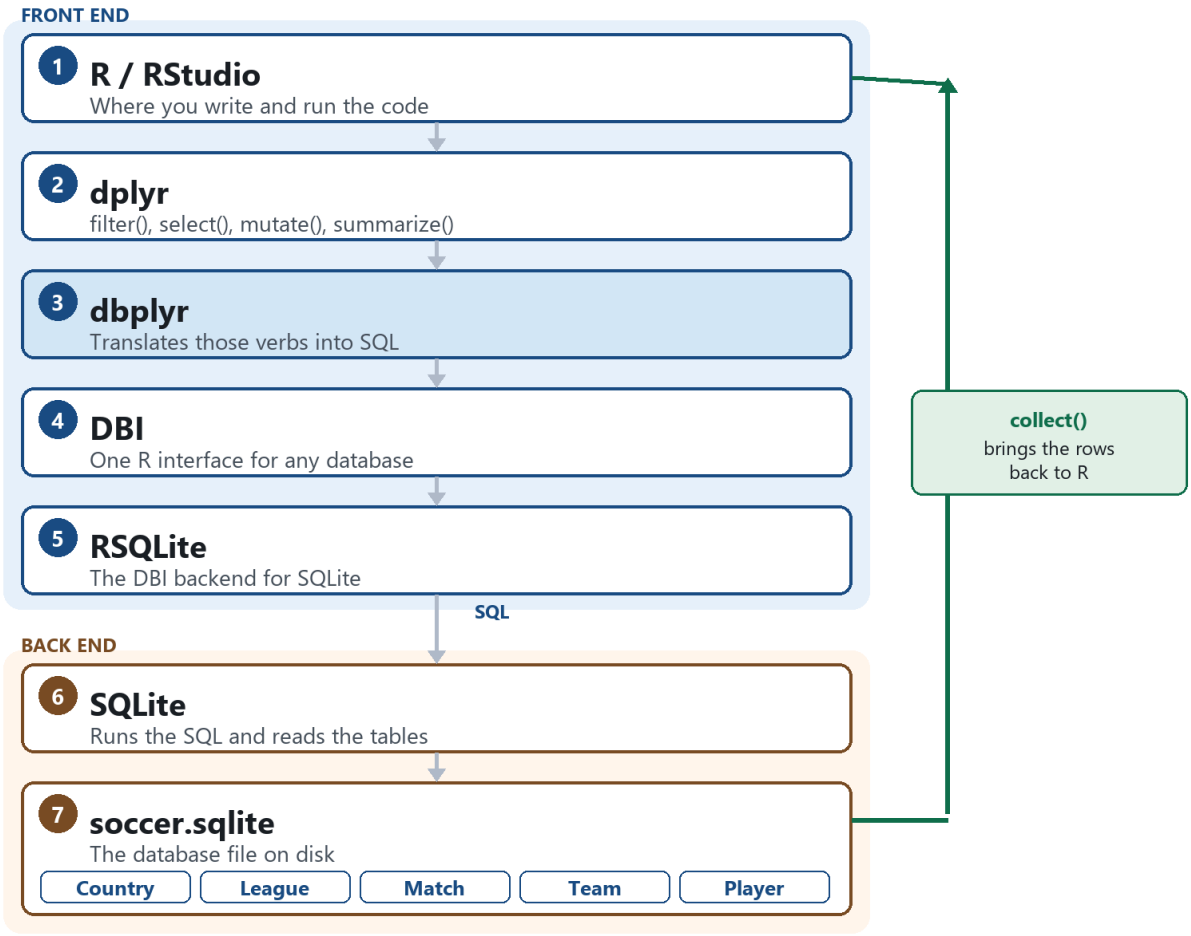
SQL is a language, not another program in the stack. `dbplyr` writes it. SQLite runs it.

Other packages play the role of `RSQLite` for other systems: `RMariaDB`, `RPostgres`, `odbc`, and `bigrquery`. You still use DBI and, usually, dbplyr. Only the backend changes.

---

4. From the front end to the back end

Read the picture from top to bottom. That is the path of a request. The green path is the way back.



The front end is what you touch in R. The back end is the database program and the file on disk.

- **R / RStudio.** You write the code here.
- **dplyr.** You say `filter()`, `select()`, `group_by()`, `summarize()`.
- **dbplyr.** It turns that pipeline into SQL. This is the step students miss. dplyr does not talk to the file itself.
- **DBI.** One set of functions for connecting, listing tables, and sending queries. The same calls work for SQLite and for other databases.
- **RSQLite.** The SQLite-specific piece of DBI. It knows how to open a `.sqlite` file.
- **SQLite.** The engine. It runs the SQL.
- **soccer.sqlite.** The file. `Match`, `Team`, `Player`, `League`, and `Country` live here, not in R's memory.

`collect()` is the return trip. SQLite has already done the work. Only the result comes back into R.

---

5. Work through the soccer database

The file in this example is `./data/soccer/soccer.sqlite`, inside the project.

Step 1. Load the packages.

```
library(DBI)
library(RSQLite)
library(dplyr)
library(dbplyr)
```

Step 2. Connect. `dbConnect()` opens the file. `con` is the connection, not the data.

```
con <- dbConnect(
  SQLite(),
  dbname = "./data/soccer/soccer.sqlite"
)
```

Step 3. See the tables.

```
dbListTables(con)
```

Step 4. Point at `Match`. `tbl()` does not download `Match`. `Match_db` is a lazy reference: a promise of a query, not a data frame in RAM.

```
Match_db <- tbl(con, "Match")
```

Step 5. Use dplyr. These verbs are recorded. SQLite has not run them yet.

```
result <- Match_db |>
  filter(season == "2010/2011") |>
  select(date, home_team_goal, away_team_goal)
```

Step 6. Read the SQL. `show_query()` prints the translation dbplyr will send.

```
show_query(result)
```

The SQL looks like this:

```
SELECT date, home_team_goal, away_team_goal
FROM Match
WHERE season = '2010/2011'
```

Step 7. Bring the rows into R. `collect()` runs the query and returns a tibble.

```
matches <- result |>
  collect()
```

`matches` is an ordinary tibble in memory. `Match_db` and `result` are still just queries.

Step 8. Close the connection.

```
dbDisconnect(con)
```

Close it yourself. If you forget, R closes it when the session ends. A file-backed database can stay locked until you do.

The same pattern works without `soccer.sqlite`. `dbConnect(SQLite(), ":memory:")` builds a database that lasts for this session, and `copy_to()` writes a data frame such as `nycflights13::flights` into it. Then `tbl()`, dplyr, `show_query()`, and `collect()` are unchanged.

---

6. Why `collect()` matters

Before `collect()`. The query stays on the database. `filter()`, `select()`, joins, and `summarize()` run in SQLite when the query is executed. Printing the lazy table shows a small preview, not the whole table.

After `collect()`. The selected rows are a tibble in RAM. Later verbs run in R. ggplot2 needs that local tibble, so plot only after `collect()`.

Not every tidyverse verb has a SQL translation. `pivot_longer()` and most tidyr helpers need a local data frame. `filter()`, `select()`, `mutate()`, `arrange()`, `group_by()`, `summarize()`, and the joins usually translate.

The point of dbplyr is to filter, join, and summarize inside the database, then transfer only the result.

---

7. The path, in one line

R, then dplyr, then dbplyr, then DBI and RSQLite, then SQLite, then `soccer.sqlite`.

- R is where you work.
- dplyr is how you say the manipulation.
- dbplyr turns that into SQL.
- DBI and RSQLite open the connection and send the query.
- SQLite does the computation.
- `soccer.sqlite` stores the tables.
- `collect()` brings the result back.

You use ordinary dplyr. You do not load the whole database into memory.

---

8. Practice

1. Name the seven stops from R to `soccer.sqlite`. Which stop is a language, not a program?
2. Connect to `soccer.sqlite`. From `Match`, keep the 2010/2011 season and the columns `date`, `home_team_goal`, and `away_team_goal`. Run `show_query()`, then `collect()`. How is `matches` different from `Match_db`?
3. Disconnect. If you do not have the soccer file, do the same steps with `nycflights13::flights` copied into an in-memory SQLite database, and filter to `month == 1`.